Отчёт по лабораторной работе № 1

«Пайплайны обработки данных на основе LangChain и LLM»

Цель работы: изучить построение цепочек обработки запросов с помощью LangChain и LCEL, преобразование естественного языка в структурированные параметры с Pydantic, взаимодействие LLM с локальной базой SQLite, обработку ошибок и сохранение состояния диалога.

Модель: **qwen3.8-27b**

Сервер: **http://10.32.2.11:35000/v1**

Основные библиотеки: LangChain, langchain-openai, Pydantic, SQLite.

1. Базовая цепочка LangChain

Для обработки запросов создан шаблон **ChatPromptTemplate** с системным и пользовательским сообщениями. В качестве системной инструкции задано требование отвечать на указанном языке. Затем с помощью LCEL собрана цепочка:

ChatPromptTemplate → ChatOpenAI → StrOutputParser.

Проверены два способа обращения к модели:

- прямой вызов **model.invoke()** возвращает объект **AIMessage**;
- вызов цепочки **chain.invoke()** после **StrOutputParser** возвращает строку **str**.

Метод **batch()** проверен на трёх запросах: о NLP, больших языковых моделях и токенизации текста.

Вывод: создана и проверена базовая LCEL-цепочка, а также рассмотрены различия между прямым вызовом модели и вызовом цепочки.

2. Structured Output и Pydantic

2.1. Модель параметров поиска

Создана Pydantic-модель **ProductQuery** со следующими полями:

Поле	Тип и ограничения
category	Enum: laptop, smartphone, tablet, other
max_price	int или None, значение не меньше 0
sort_by	Enum: price_asc или price_desc

Для разбора ответа используется **PydanticOutputParser**. Его инструкции формата добавляются в системный промпт. Для каждого запроса сохранялись исходная формулировка, объект Pydantic и JSON-представление.

Проверены семь запросов: пять основных и два неоднозначных. В финальном запуске получены следующие параметры:

Запрос	Категория	Максимальная цена	Сортировка
Ноутбуки дешевле 70 000 ₽	laptop	70 000	price_asc
Самые дорогие смартфоны	smartphone	не задана	price_desc

Запрос	Категория	Максимальная цена	Сортировка
Планшеты без ограничения цены	tablet	не задана	price_asc
Ноутбук до 50 000 ₽	laptop	50 000	price_asc
Холодильники	other	не задана	price_asc
Устройство для работы, которое удобно брать с собой	laptop	не задана	price_asc
Гаджет для чтения книг и просмотра фильмов, желательно недорогой	tablet	не задана	price_asc

Для последнего запроса модель не стала самостоятельно придумывать конкретное значение бюджета: поле **max_price** осталось равным **None**.

2.2. Сравнение thinking OFF и thinking ON

Для двух неоднозначных запросов выполнены вызовы одной модели с выключенным и включённым режимом thinking. Измерялись время выполнения и количество токенов. Все четыре ответа успешно разобраны через Pydantic-парсер.

Запрос	Режим	Время, с	Входные токены	Выходные токены	Результат
Устройство для работы и переноски	OFF	0,31	430	31	laptop
Устройство для работы и переноски	ON	8,67	466	753	other
Чтение книг и просмотр фильмов	OFF	0,34	432	35	tablet
Чтение книг и просмотр фильмов	ON	1,52	468	185	tablet

При включённом thinking число выходных токенов увеличилось примерно в 24,3 раза для первого запроса и в 5,3 раза для второго. Время ответа также выросло. Для первого запроса результат различался: OFF выбрал **laptop**, ON — **other**. Для второго запроса оба режима выбрали **tablet**.

В доступном содержимом ответов отображался JSON; отдельный текст рассуждений в **content** не выводился. В **additional_kwargs** для ответов с thinking было указано **refusal: None**. По этим данным нельзя делать вывод о внутреннем отсутствии рассуждений — можно утверждать только, что отдельный текст рассуждений не был представлен в доступном содержимом ответа.

Вывод: включение thinking в данном эксперименте увеличило задержку и число выходных токенов, но не обеспечило одинаково более подходящий результат для всех запросов. Для преобразования запросов в компактные параметры поиска режим OFF оказался экономичнее.

3. Взаимодействие LLM с SQLite

3.1. База данных и функция поиска

Создана SQLite-таблица **products** с полями **id**, **name**, **category**, **price**. В тестовую базу добавлено 16 записей:

- 5 ноутбуков;
- 5 смартфонов;
- 5 планшетов;

- 1 товар категории **other** (холодильник).

Функция **search_products(product_query)** принимает объект **ProductQuery**, формирует SQL-запрос и использует параметры **?** для подстановки значений. Поддерживаются фильтрация по категории и максимальной цене, а также сортировка по возрастанию или убыванию цены. Категория **other** обрабатывается как неподдерживаемая.

Проведено 11 проверок, включая:

- поиск по каждой из трёх основных категорий;
- фильтрацию по максимальной цене;
- сортировку в обоих направлениях;
- пустой результат (планшеты дешевле 10 000 ₽);
- неподдерживаемую категорию **other**.

Все проверки дали ожидаемый результат.

3.2. Объединение LLM и SQLite через LCEL

Создана цепочка:

Prompt → LLM → PydanticOutputParser → RunnableLambda(search_products).

Проверены четыре запроса:

- ноутбуки дешевле 70 000 ₽ — найдено 4 товара;
- самые дорогие смартфоны — найдено 5 товаров в порядке убывания цены;
- планшеты дешевле 20 000 ₽ — найден 1 товар;
- холодильники — возвращено сообщение о неподдерживаемой категории.

Для первых трёх запросов отдельно получены параметры от LLM без обращения к базе. Это показывает разницу между извлечением параметров и фактическим поиском: сама модель не получает содержимое SQLite, а конкретные товары возвращает программная функция.

3.3. Оформление поиска как инструмента

Функция поиска оформлена как инструмент **search_products** с помощью **@tool** и **args_schema=ProductQuery**. Инструмент подключён к модели через **bind_tools()**.

Проверены семь запросов. Вызовы инструмента и их аргументы проверялись через **ProductQuery** до выполнения SQL-поиска.

Наблюдения:

- для обычных запросов модель формировала подходящие аргументы;
- для запроса о холодильниках в одном из тестов модель не вызвала инструмент;
- для неоднозначного запроса об устройстве для работы модель сформировала два вызова — с категориями **tablet** и **laptop**;
- для запроса о чтении книг и просмотре фильмов была выбрана категория **tablet**, но числовой бюджет не задан;
- в одном из вызовов модель передала строку **"None"** вместо значения **null**. Pydantic отклонил аргумент, поэтому SQL-поиск не был выполнен с некорректными параметрами.

Вывод: tool calling позволяет модели инициировать вызов программной функции, но не гарантирует правильную интерпретацию неоднозначного запроса. Проверка аргументов перед обращением к базе обязательна. Отсутствие вызова инструмента также должно обрабатываться отдельно.

4. Обработка ошибок и повторные попытки

Проверены три вида некорректных ответов:

1. строка, не являющаяся JSON;
2. недопустимое значение категории **camera**;
3. отрицательная цена **-1000**, нарушающая ограничение **ge=0**.

Во всех случаях **PydanticOutputParser** выдавал **OutputParserException**. Внутри сообщений об ошибке отражалась исходная причина: неправильный формат JSON или нарушение Pydantic-валидации.

Для проверки повторных попыток использован **FakeListChatModel**:

- в успешном сценарии первая генерация вернула неправильный JSON, вторая — корректный объект **ProductQuery**;
- в неуспешном сценарии все три ответа были некорректными, после чего цепочка вернула **OutputParserException**;
- при сравнении без retry цепочка завершилась ошибкой после первой попытки, а с retry получила корректный результат со второй попытки.

Повтор выполнялся для цепочки генерации и парсинга. Поиск в SQLite не запускался до успешной проверки параметров.

Вывод: **with_retry(stop_after_attempt=3)** помогает восстановиться после временной ошибки формата, но не гарантирует успех, если все попытки возвращают некорректные данные.

5. Сохранение состояния диалога

Создана цепочка, которая получает текущие параметры поиска и новое сообщение пользователя. После каждого уточнения результат парсинга сохраняется в переменную **current_query** вне модели.

Проверены два диалога.

Диалог 1:

1. ноутбуки до 70 000 ₽;
2. уточнение: до 50 000 ₽;
3. уточнение: сортировать от самых дорогих.

Итог: **laptop**, **max_price=50000**, **price_desc**.

Диалог 2:

1. планшеты без ограничения цены;
2. уточнение: сначала самые дорогие;
3. уточнение: теперь смартфоны.

Итог: **smartphone**, **max_price=None**, **price_desc**.

В обоих диалогах неизменённые параметры сохранялись корректно.

5.1. Сравнение полной истории и хранения состояния

Для двух последних ходов сравнивались два подхода: передача модели полной истории диалога и передача только актуального объекта **ProductQuery** вместе с новым сообщением.

Диалог	Полная история, входные токены	Хранение состояния, входные токены	Экономия
1	479	434	45
2	457	426	31
Итого	**936**	**860**	**76**

Суммарная экономия составила около 8,1%. Оба подхода вернули одинаковые и корректные параметры в каждом из двух тестов.

Вывод: хранение актуального состояния позволило сохранить корректность уточнений и уменьшить количество входных токенов в проведённых экспериментах.

6. Общие выводы

В лабораторной работе реализованы все основные этапы конвейера обработки запросов:

- создание LCEL-цепочки для общения с LLM;
- преобразование естественного языка в структурированные данные с Pydantic;
- сравнение режимов thinking OFF и ON;
- выполнение параметризованных SQL-запросов к SQLite;
- подключение поиска как инструмента модели;
- обработка некорректных ответов и повторная генерация;
- хранение состояния диалога вне модели.

Эксперименты показали, что структурированная валидация необходима перед выполнением программных действий. Подключение инструмента не исключает неоднозначности и ошибок в аргументах, поэтому нужны проверки и обработка отсутствующего вызова. Хранение текущего состояния диалога в протестированных примерах обеспечило те же результаты, что и передача полной истории, при меньшем количестве входных токенов.

7. Ограничения эксперимента

Результаты измерений относятся к конкретным запускам модели **qwen3.8-27b** и могут меняться при повторном выполнении из-за особенностей генерации и нагрузки сервера. В части 5 сравнивались два коротких тестовых диалога, поэтому полученную экономию токенов нельзя автоматически переносить на диалоги любой длины. База содержит демонстрационные товары и предназначена для проверки логики поиска, а не для оценки реального ассортимента.